pfi

# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

# From Notebook to Module

WEEK 4 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Why a module

The September cover note, reconciled in week 3, day 3

| 200 | 489,000,000 | 490,819,215 |
| --- | --- | --- |
| bonds | dollars of face | dollars of market value |

### A notebook is where work is found

Cells run one at a time, with the output under each.

The place to look at the data and make each decision.

A function in a cell lives in that notebook only.

### A module is where work is kept

A plain text file ending in .py that holds functions.

Any notebook can import it: one copy, one fix.

The same steps run on next month's file.

**Next month a new extract arrives. The decisions have not changed, so they are written down once, with their checks.**

# desk_tools.py: five functions

| load_holdings | clean_holdings | check_totals | summarize | write_report |
|---|---|---|---|---|
| Reads the CSV and returns it as it arrived | Applies the six decisions of week 3, day 3 | Tests the table against the cover note | Returns the headline numbers, one row | Writes an Excel workbook, two sheets |
| in: path | in: raw | in: book and the three totals | in: book | in: book, summary, path |
| **out: raw** | **out: book** | **out: nothing, or a ValueError** | **out: summary** | **out: a file** |

```
raw = desk_tools.load_holdings(data_folder + file)
book = desk_tools.clean_holdings(raw)
desk_tools.check_totals(book, expected_bonds, expected_face, expected_value)
summary = desk_tools.summarize(book)
desk_tools.write_report(book, summary, report_file)
```

**Everything a function needs comes in through its parameters. Everything it produces goes out through return.**

Every line of pandas inside them was taught in an earlier notebook. What is new is where the lines live.

# Writing it and importing it

## Write the file

```
%%writefile desk_tools.py
import pandas as pd

def load_holdings(path: str):
    ...
Writing desk_tools.py
```

107 lines, 5 functions. Nothing in the cell ran: it saved a text file. The file has its own import of pandas, and does not see the notebook's variables.

## Import it, and ask it

```
import desk_tools
dir(desk_tools)
help(desk_tools.clean_holdings)
desk_tools.load_holdings(...)
```

dir() lists the names in the module. help() prints the signature and the docstring, word for word. The docstring is the documentation.

## Type hints: read them

```
def load_holdings(path: str)
        -> pd.DataFrame:
```

path: str says the function expects text. The arrow says it returns a DataFrame. Python does not enforce a hint. This course asks you to read them, not write them.

**Each function opens with a docstring: what goes in, what comes out, and the units.**

### Excel side by side

The nearest comparison is a workbook of calculations that other workbooks refer to. It is a loose one: a module holds steps and no data.

# A check that raises

```
def check_totals(book, bonds, face, market_value):
    found_bonds = len(book)
    problems = []
    if found_bonds != bonds:
        problems.append(f'{found_bonds:,} rows against {bonds:,} bonds')
    # the same for the face and for the market value
    if problems:
        raise ValueError('control totals do not match: '
                         + '; '.join(problems))
```

**A printed difference can be scrolled past. An error cannot.**

### On the extract as received: Predict 1

ValueError: control totals do not match: 204 rows against 200 bonds; face of 495,500,000 against 489,000,000 dollars; market value of 497,769,925.00 against 490,819,215.00 dollars

### On the cleaned table

No error and no output. Silence means the table matches: 200 bonds, 489,000,000 dollars of face, 490,819,215.00 dollars of market value.

### In Excel, on a sheet holding the cleaned book: CUSIP in column B, face in R, market value in S

`=COUNTA(B2:B201)` returns 200 `=SUM(R2:R201)` returns 489,000,000 `=SUM(S2:S201)` returns 490,819,215

`=IF(COUNTA(B2:B201)=200, "OK", "CHECK")` shows the result in a cell, and the reader has to look at it.

**check_totals stops the run, so no report is written from a table that does not match.**

# Edit, then reload

### desk_tools.py, on disk

6 functions, after %%writefile -a added count_changes.

reload →

### The copy in memory

5 functions. import ran the file once, in section 19.3.

```
changes = desk_tools.count_changes(raw, book)
AttributeError: module 'desk_tools' has no attribute 'count_changes'

importlib.reload(desk_tools)     # runs the file again, top to bottom
changes = desk_tools.count_changes(raw, book)
```

**After every edit to a .py file: reload it, or restart the kernel and run from the top.**

Running import desk_tools a second time does nothing: the module is already imported.

count_changes on the September extract

| Decision | Rows |
|---|---|
| rows_removed | 4 |
| tickers_tidied | 8 |
| stale_prices_flagged | 6 |
| ratings_labelled | 5 |
| yields_corrected | 2 |

The counts week 3 found one section at a time, in the order of its decision log.

# The same five lines, two files

```
for file in ['holdings_messy.csv', 'holdings.csv']:
    raw_file = desk_tools.load_holdings(data_folder + file)
    book_file = desk_tools.clean_holdings(raw_file)
    desk_tools.check_totals(book_file, expected_bonds, expected_face, expected_value)
    summaries[file] = desk_tools.summarize(book_file)
    changes[file] = desk_tools.count_changes(raw_file, book_file)
```

**The only thing that changed between the two runs was the file name.**

Both cleaned tables pass check_totals: 200 bonds, 489,000,000 dollars of face, 490,819,215 dollars of market value, a mean OAS of 185.75 basis points, and a mean yield of 6.088 percent.

What still differs between the two cleaned books: rating in 5 rows, and price_date and stale_price in 6. The other 17 of the 20 columns are identical. Those are open items for operations.

| | holdings_messy.csv | holdings.csv |
|---|---|---|
| As received | raises ValueError | matches |
| Rows in | 204 | 200 |
| rows_removed | 4 | 0 |
| tickers_tidied | 8 | 0 |
| stale_prices_flagged | 6 | 0 |
| ratings_labelled | 5 | 0 |
| yields_corrected | 2 | 0 |
| Rows out | 200 | 200 |
| After cleaning | matches | matches |

**Predict 3: on the clean file, every count is 0.**

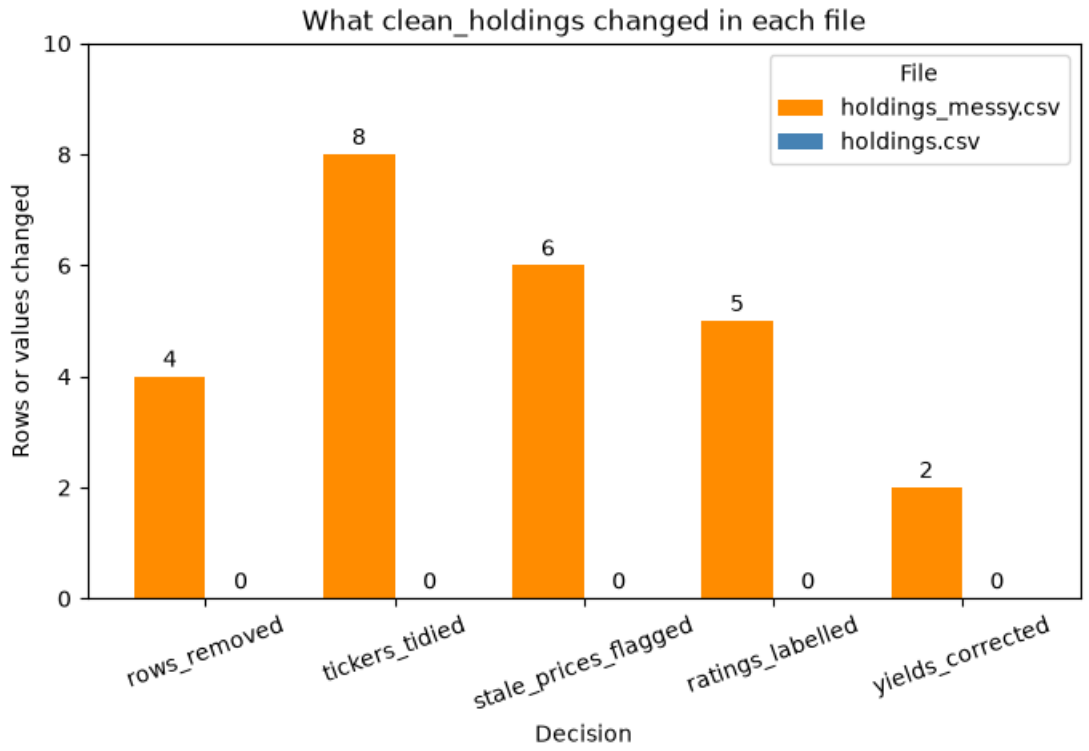
IQR lower limit there: 3.055 percent. Lowest yield: 4.089.

### Excel side by side

Paste the new rows in, then check every range: S2:S205 covers the 204 rows of one extract, S2:S201 the 200 rows of the other.

# What the routine changed

Rows or values changed by clean_holdings, in each file




## A routine that is safe to keep

It fixes the file that has problems, and it leaves the file that has none alone. The empty blue bars are the evidence of the second.

all_changes.T swaps rows and columns, so each decision is a group of bars. In Excel: Paste Special, Transpose.

```
all_changes.T.plot.bar(ax=ax,
    color=['darkorange', 'steelblue'])
ax.bar_label(container, fmt='{:.0f}')
```

# The report workbook

```
def write_report(book, summary, path):
    with pd.ExcelWriter(path) as writer:
        summary.to_excel(writer, sheet_name='summary', index=False)
        book.to_excel(writer, sheet_name='book', index=False)
```

```
book_back = pd.read_excel(report_file, sheet_name='book',
                          dtype={'cusip': str})
desk_tools.check_totals(book_back, 200, 489_000_000, 490_819_215)
```

The workbook holds values, not formulas. Read back from the file, the book sheet still passes check_totals. The notebook then deletes the file: on a desk it would be kept and sent.

**Sheet summary: 1 row, 7 columns**

| Column | Value |
|---|---|
| bonds | 200 |
| face | 489,000,000 |
| market_value | 490,819,215 |
| mean_oas | 185.75 |
| mean_yield | 6.088 |
| not_rated | 5 |
| stale_prices | 6 |

**Sheet book: 200 rows, 20 columns**

The 19 columns of the extract, and stale_price.

| Job | pandas | Excel |
|---|---|---|
| Two tables in one file | pd.ExcelWriter(path), one to_excel() for each | one workbook, two sheets |
| Read one sheet | pd.read_excel(path, sheet_name='summary') | select the summary tab |
| Control totals | desk_tools.check_totals(book_back, ...) | =COUNTA(book!B2:B201), =SUM(book!R2:R201) |

# Testing a function with assert

A table typed by hand: seven rows, one of each problem

| row | cusip | ticker | rating | yield_pct | price_date |
|---|---|---|---|---|---|
| 0 | TEST00001 | KRIX | BBB | 5.1 | 2026-09-30 |
| 1 | TEST00002 | krix | BBB | 6.2 | 2026-09-30 |
| 2 | TEST00003 | MORX | missing | 5.5 | 2026-09-30 |
| 3 | TEST00004 | MORX | A | 4.9 | 2026-08-14 |
| 4 | TEST00005 | THAA | BB | 0.0575 | 2026-09-30 |
| 5 | TEST00006 | CORA | B | 5.8 | 2026-09-30 |
| 6 | TEST00006 | CORA | B | 5.8 | 2026-09-30 |

Worked out by hand first: 6 rows, the ticker KRIX in row 1, Not rated in row 2, a stale price in row 3, and 5.75 in row 4.

```
def test_clean_holdings():
    result = desk_tools.clean_holdings(tiny)
    assert len(result) == 6, f'expected 6 rows, found {len(result)}'
    assert result.loc[2, 'rating'] == 'Not rated', 'missing rating not labelled'
    # three more: the tickers, the yields, the stale price
    print('clean_holdings: 5 checks passed')

test_clean_holdings()
clean_holdings: 5 checks passed
```

Control totals test a run. A test checks the function: tickers are in no total, so only a test would see a broken ticker step. Run it after every edit, straight after the reload.

**pytest runs every test_ function with one command.**

### A test that fails: the colleague expected the label NR

`assert tidy.loc[2, 'rating'] == 'NR'  ->  AssertionError: expected the label NR, found Not rated`

The function and the test disagree. The docstring settles it: decision 4 says the label is Not rated.

In Excel: a small sheet of made-up rows with known answers, to try a formula on before trusting it on 200 rows.

# When not to use Python

## Stay in a spreadsheet for

A one-off look at a file you will not see again

A table someone else must edit by hand

A job with no repetition

A file too small to matter

A spreadsheet task is worth moving when several of these are true

| Sign | What it looks like |
|---|---|
| It repeats | the same steps every day, week, or month end |
| The input changes shape | a different number of rows, so ranges are re-pointed |
| It has control totals | a figure from outside the file that the result must match |
| Others rely on it | the result goes into a pack, and a mistake would travel |
| The steps are hard to see | nobody can say what was done without clicking cells |

**The two tools work together: the module did the repeatable part, and the output was a workbook.**

## A line you will meet in other people's files

```
if __name__ == '__main__':
```

The lines under it run only when the file is run as a program, with python desk_tools.py or %run, and not when it is imported. The functions above it are the module.

# The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

## Read

Read the code before you run it.

2

## Predict

Decide what it should produce.

3

## Run

Run it and look at the output.

4

## Check

Compare with a number you worked out yourself.

*Code that runs is not the same as code that is right.*

# What you can do now

- Write a .py file of functions with docstrings, and import it
- Read a module with dir() and help(), and reload it after an edit
- Write a check that raises, with a message that names every mismatch
- Run the same lines on a second file by changing the file name
- Test a function with assert, on a small table typed by hand

**Week 4, Day 5** The Course Project. A new extract, and a cleaning function you write yourself